

AI in Brain Tumor Diagnosis

Classification and care



Under Supervision Of :

Prof . Amany Sarhan

Noura Shousha | Esraa Alaa
Mohamed El-Gazar | Mahmoud El-Gendy



TABLE OF CONTENTS

01

INTRODUCTION

02

PROBLEM

03

SOLUTION

04

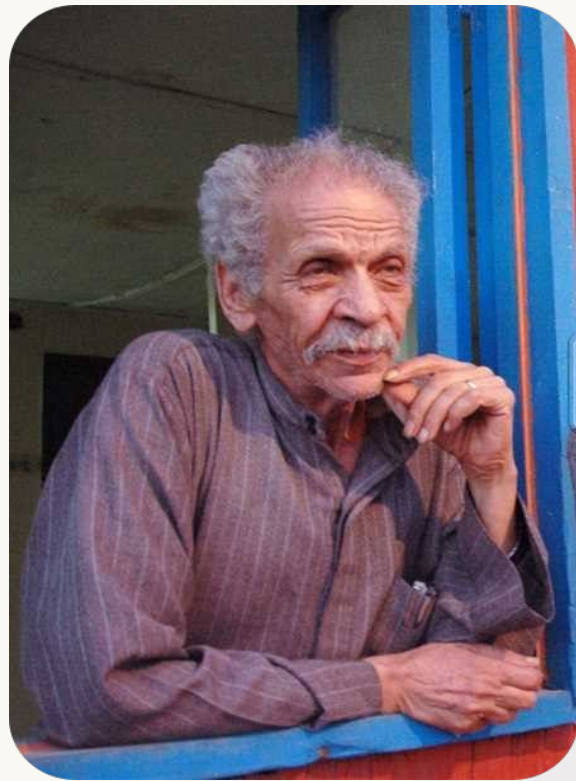
RESULTS

05

FUTURE WORK

06

CONCLUSIONS





PROBLEM



1M
Affected

90,000
New/Y

19,000
Death /
Y

8740
Egy-202
5

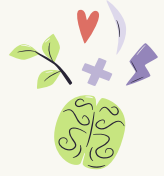




Early Detected
Can survive up to
20 years



Late Detected
Can survive up to
5 years





SOLUTION



OUR WORK



FOR EXPERTS:

**DETECTION
AND
CLASSIFICATION**

FOR PATIENTS:

**MEDICAL
CHATBOT**

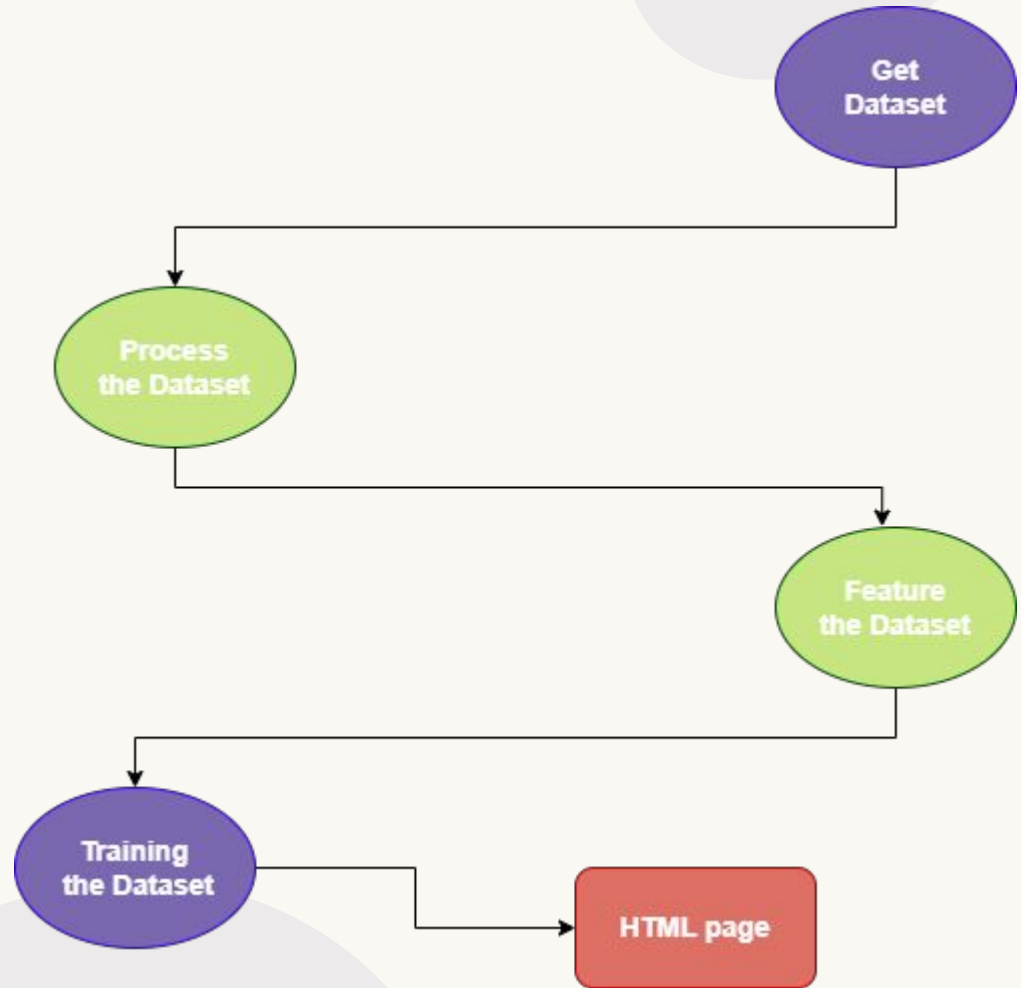
01

DETECTION AND CLASSIFICATION N



Training :

- Data Collection
- Data processing
- Feature Extraction
- Model Selection
- Model Training
- Model Evaluation
- Model Testing
- Fine-Tuning and Optimization



WE USED:

Some of the most important python libraries were used, such as:

- Os
- Requests
- Flask
- PIL
- TF
- Pytorch



```
1 import os
2 import requests
3 from flask import Flask, flash, request, redirect, url_for, render_template, session
4 from werkzeug.utils import secure_filename
5 from torchvision import transforms, models
6 import torch
7 from PIL import Image
8 import cv2
9 from tensorflow.keras import models as TF
10 import numpy as np
11
12 UPLOAD_FOLDER = './static/images'
13 ALLOWED_EXTENSIONS = ['png', 'jpg', 'jpeg']
14 LABELS = ['None', 'Meningioma', 'Glioma', 'Pituitary']
15 DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
16
17 app = Flask(__name__, template_folder='template')
18 app.config['UPLOAD_FOLDER'] = UPLOAD_FOLDER
19 app.secret_key = "m4xpl0it"
```

```
1 def load_model():
2     resnet_model = models.resnet50(weights=True)
3
4     for param in resnet_model.parameters():
5         param.requires_grad = True
6
7     n_inputs = resnet_model.fc.in_features
8
9     resnet_model.fc = torch.nn.Sequential(torch.nn.Linear(n_inputs, 2048),
10                                           torch.nn.SELU(),
11                                           torch.nn.Dropout(p=0.4),
12                                           torch.nn.Linear(2048, 2048),
13                                           torch.nn.SELU(),
14                                           torch.nn.Dropout(p=0.4),
15                                           torch.nn.Linear(2048, 4),
16                                           torch.nn.LogSigmoid())
17
18     for name, child in resnet_model.named_children():
19         for name2, params in child.named_parameters():
20             params.requires_grad = True
21
22     resnet_model.to(DEVICE)
23
24     resnet_model.load_state_dict(torch.load('./assets/bt_resnet50_model.pt', map_location=DEVICE))
25
26     resnet_model.eval()
27     return resnet_model
28
29 MODEL = load_model()
30 TF_MODEL = TF.load_model('./assets/kaggle_model.h5')
```



- Numpy
- Werkzeug libraries
- CV2
- Keras
- Pandas
- Scipy
- Matplotlib

```
1 @app.route('/empty_page')
2 def empty_page():
3     filename = session.get('filename', None)
4     os.remove(os.path.join(UPLOAD_FOLDER, filename))
5     return redirect(url_for('index'))
6
7 @app.route('/pred_page')
8 def pred_page():
9     pred = session.get('pred_label', None)
10    f_name = session.get('filename', None)
11    return render_template('pred.html', pred=pred, f_name=f_name)
12
13 def predict(image_path):
14     img = Image.open(image_path).convert('RGB')
15
16     transform = transforms.Compose([transforms.Resize((512, 512)), transforms.ToTensor()])
17
18     img = transform(img)
19
20     img = img[None, ...]
21
22     with torch.no_grad():
23         y_hat = MODEL.forward(img.to(DEVICE))
24         predicted = torch.argmax(y_hat.data, dim=1)
25         print("y_hat.data:", y_hat.data)
26         print("predicted:", predicted)
27         print("Prediction:", LABELS[predicted.data], '\n')
28     return {'class_id': str(predicted.data), 'class_name': LABELS[predicted.data]}
```

```
1 def predict_tensorflow(image_path):
2     # the tensorflow model expects BGR image not RGB
3     img = cv2.imread(image_path)
4     img = cv2.resize(img, (150,150))
5     img = img.reshape(1,150,150,3)
6     p = TF_MODEL.predict(img)
7     p = np.argmax(p,axis=1)[0]
8
9     if p==0:
10        p='Glioma'
11    elif p==1:
12        p="No Tumor"
13    elif p==2:
14        p='Meningioma'
15    else:
16        p='Pituitary'
17
18    print('The Model prediction is:', p)
19    return p
```



```
1 @app.route('/', methods=['POST', 'GET'])
2 def index():
3     try:
4         if request.method == 'POST':
5             f = request.files['bt_image']
6             filename = str(f.filename)
7             print('filename:', filename)
8             if filename != '':
9                 ext = filename.split(".")
10
11                 if ext[1] in ALLOWED_EXTENSIONS:
12                     filename = secure_filename(f.filename)
13                     save_path = os.path.join(app.config['UPLOAD_FOLDER'], filename)
14                     f.save(save_path)
15
16                     predicted = predict(save_path)
17                     tf_res = predict_tensorflow(save_path)
18                     # session['pred_label'] = f"Pytorch Model: {predicted['class_name']}<br>Tensorflow Model: {tf_res}"
19                     session['pred_label'] = [predicted['class_name'], tf_res]
20                     session['filename'] = filename
21                     return redirect(url_for('pred_page'))
22
23     except Exception as e:
24         print("Exception\n")
25         print(e, '\n')
26
27     return render_template('index.html')
28
29 if __name__ == "__main__":
30     app.run(port=3000, debug=True)
```

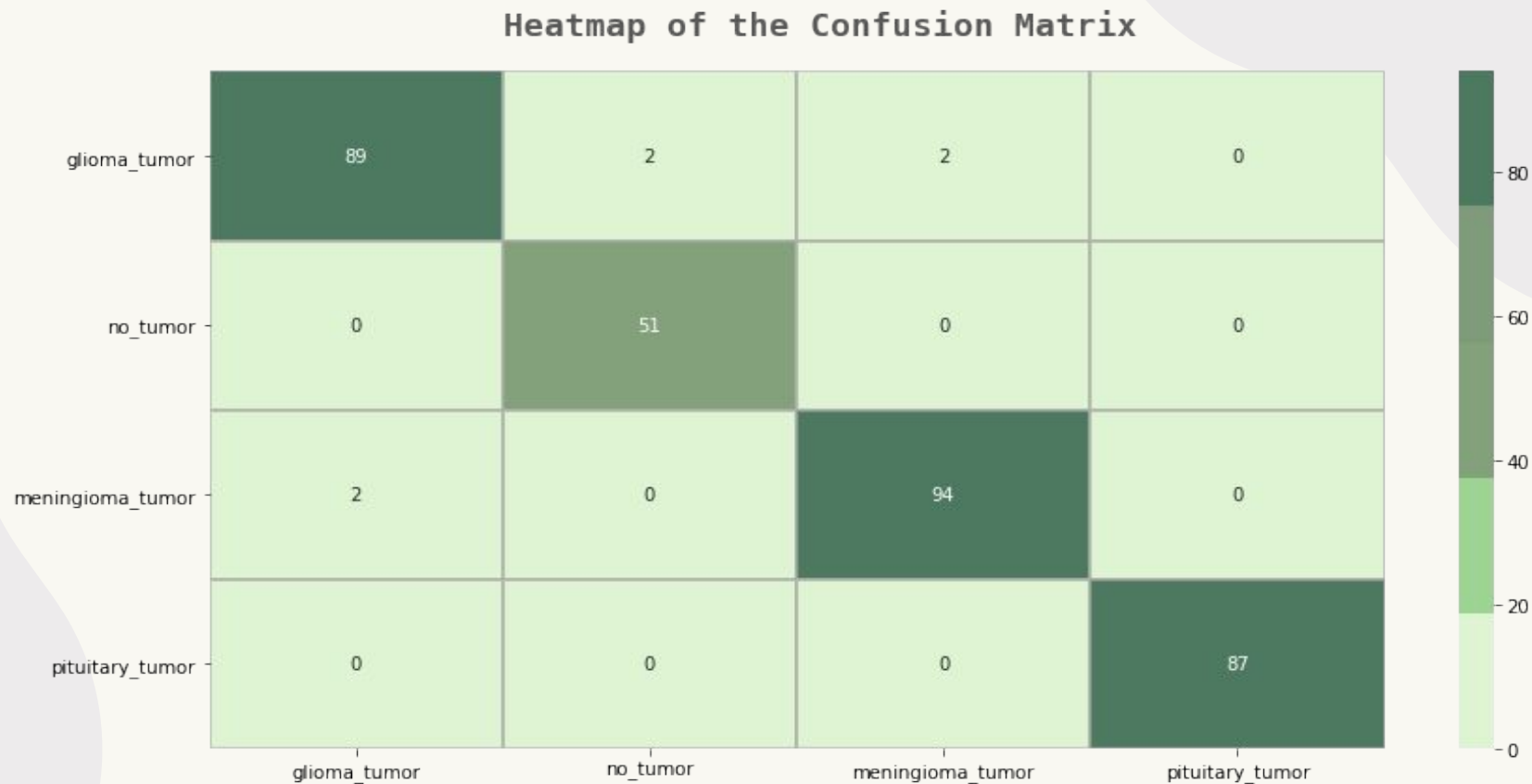


TENSORFLOW MODEL

Classification Report

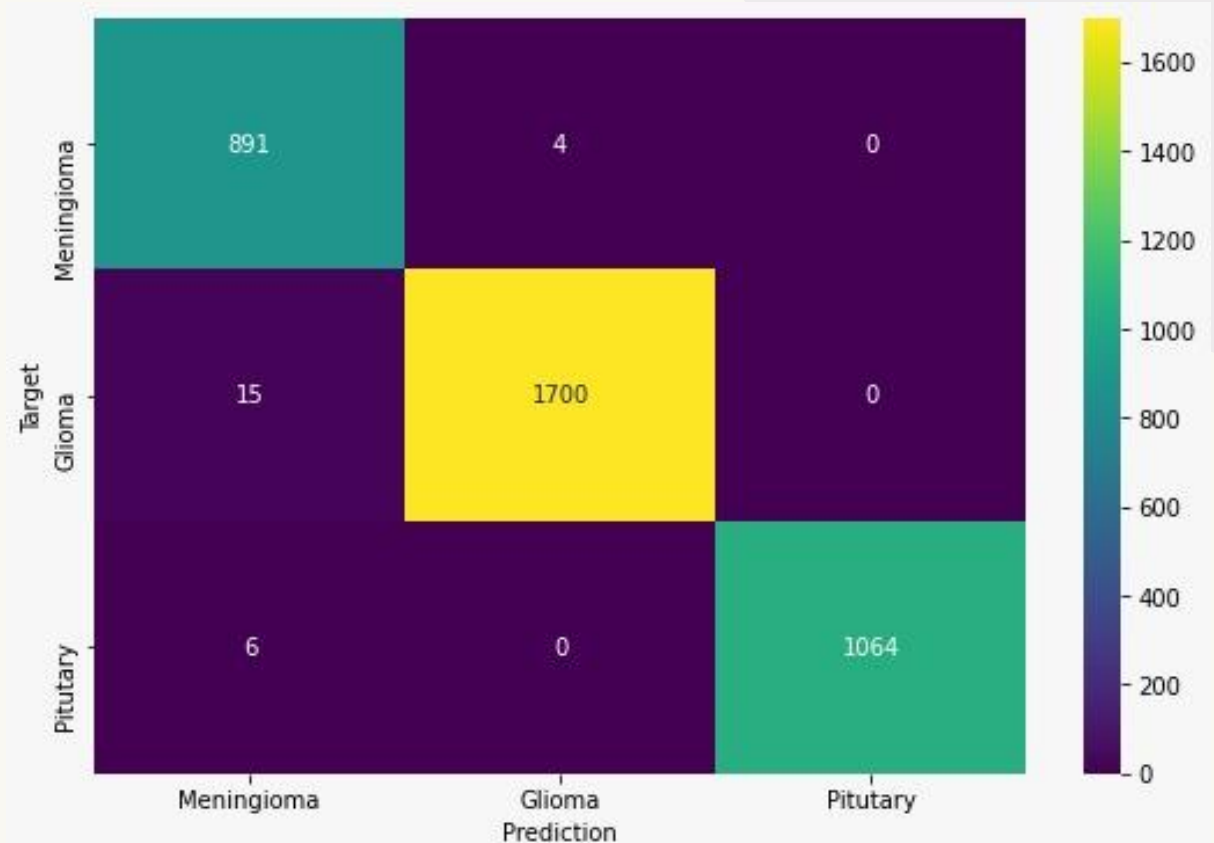
	precision	recall	f1-score	support
0	0.98	0.96	0.97	93
1	0.96	1.00	0.98	51
2	0.98	0.98	0.98	96
3	1.00	1.00	1.00	87
accuracy	-	-	0.98	327
Macro avg	0.98	0.98	0.98	327
Weighted avg	0.98	0.98	0.98	327

Heatmap of Confusion Matrix



RESNET-50 MODEL

Heatmap of
Confusion
Matrix:



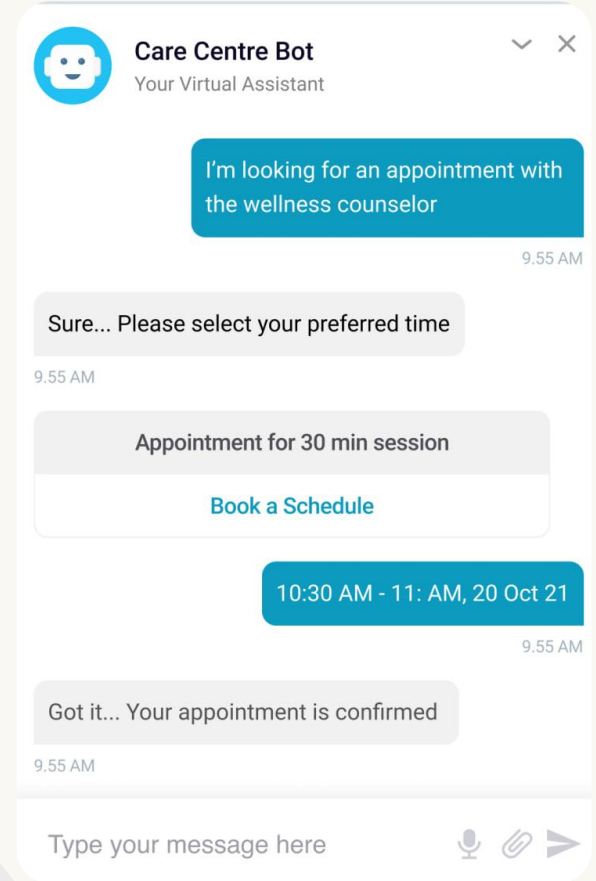
02

MEDICAL CHATBOT

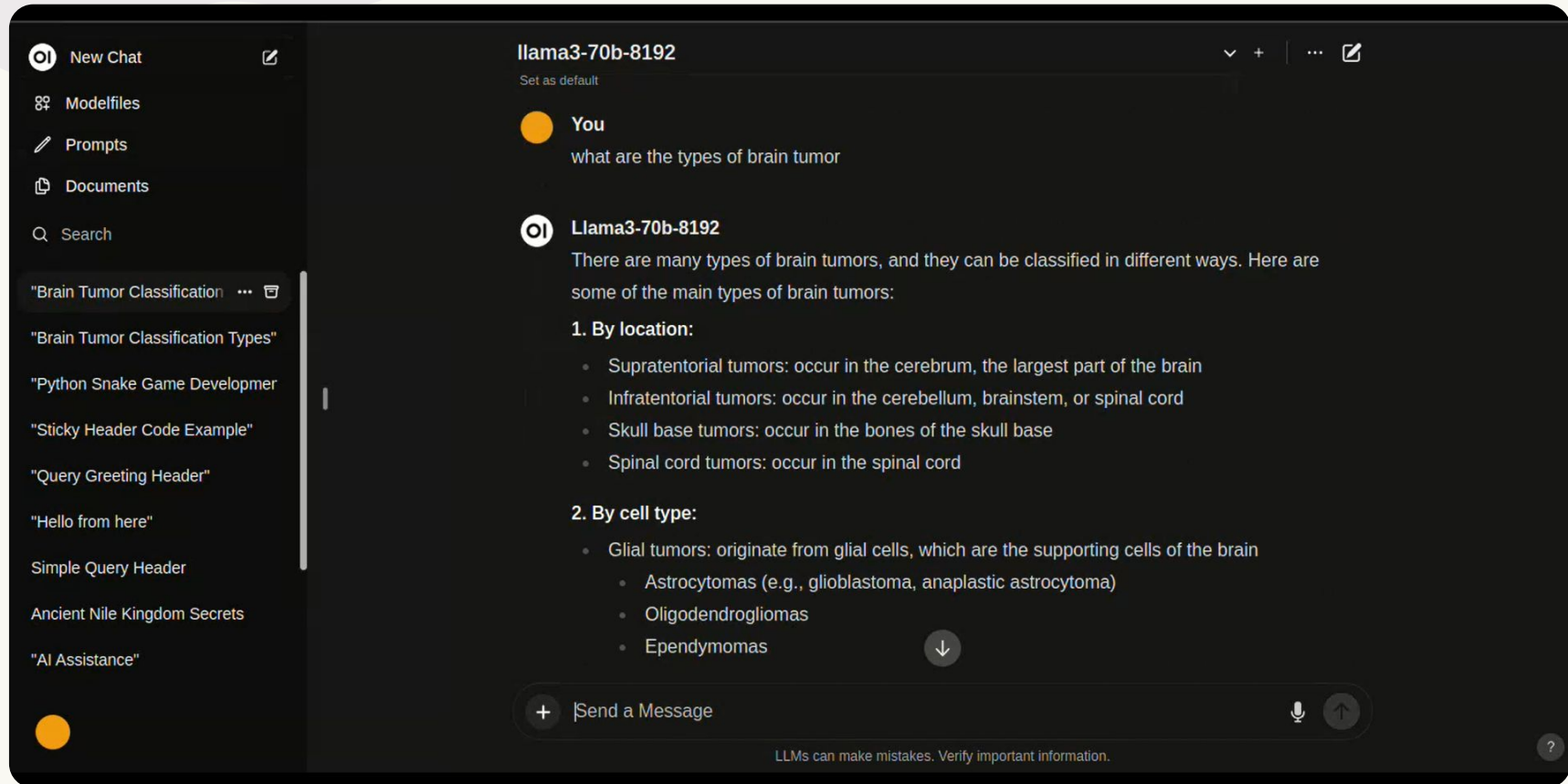


What is a Medical chatbot?

- Is a conversational AI-powered solution
- Designed to make healthcare more interactive and proactive.
- Chatbots were used also for firsthand symptom assessment.
- Healthcare chatbots act as the mediator between patients and their healthcare providers.

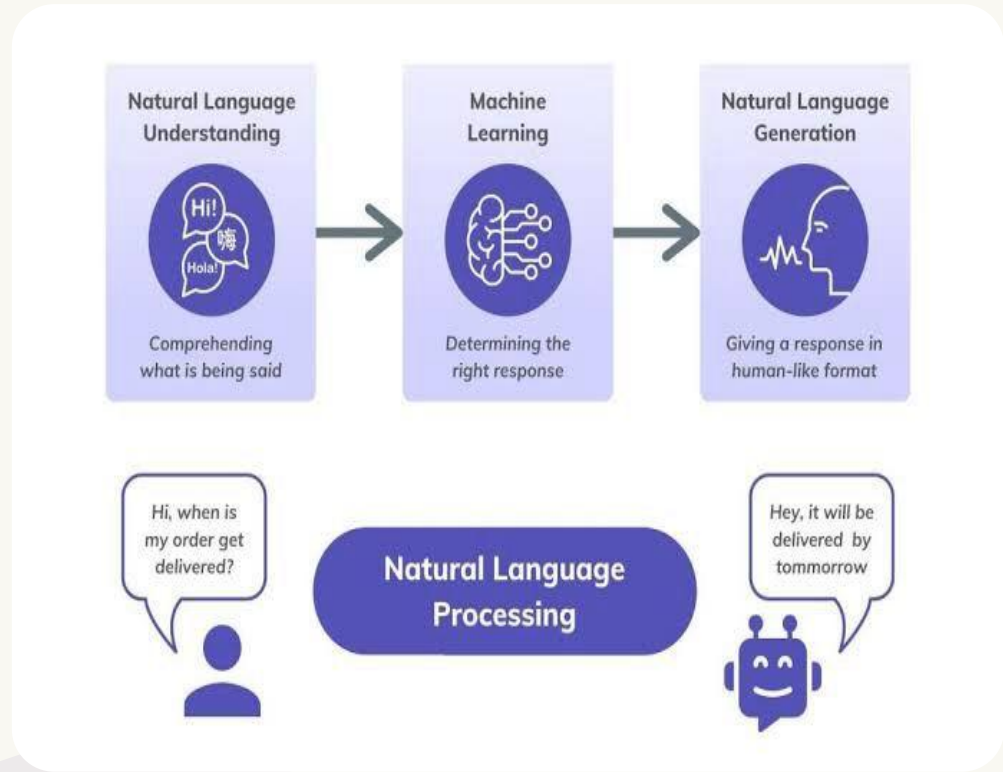


Our medical chatbot:



The relationship between NLP and CHATBOT:

- Understanding User Input
- Sentiment Analysis
- Language Translation
- Context Management
- Information Retrieval
- Named Entity Recognition



WE USED:

Some of the most important models and apps were used, such as:

- Ollama
- Docker
- Open WebUI
- Meta





Results





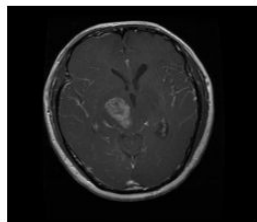
Classification Output

NeuroAI Detectives

Upload an Image

Classify

where the test image can be provided



	 TensorFlow
Glioma	Glioma

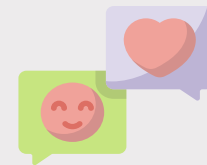
Go home

This is how the pred page is displayed

(in this case it's Glioma, there are more types to classify)



Medical



New Chat

Modelfiles

Prompts

Documents

Search

New Chat

"Greetings and Inquiries Welcome"

"Query Greeting and Salutations"

"Math Problem Resolution Needed"

"Greeting and Inquiry Phrase"

NeuroAI Detectives

llama3-70b-8192
Set as default

You
i had a brain tumor surgery and i have a bad headache can you help me ? reply shortly

llama3-70b-8192
So sorry to hear that you're experiencing a bad headache after brain tumor surgery That's not uncommon. Here are some suggestions that may help:

- Stay hydrated: Drink plenty of water to help your body recover.
- Rest: Try to get plenty of rest and avoid strenuous activities.
- Pain management: Follow your doctor's advice on pain medication and take it as directed.
- Ice pack: Apply a cold compress or ice pack to your forehead or neck to help reduce pain and swelling.

Remember to contact your doctor or neurosurgeon if your headache worsens, if you have increased nausea or vomiting, or if you experience any unusual symptoms.

Hope this helps, and feel better soon!

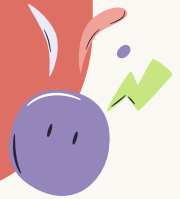
+ Send a Message

LLMs can make mistakes. Verify important information.

It helps track the patient's overall state while healing



Future Extensions





WHAT TO EXPECT ?



Personalized
Treatment



Bot powers
up
knowledge



Integration
with
Wearable
Devices



Clinical
Validation





Conclusion



How exactly is this project helpful?



1) Assisting healthcare professionals ,by:

- Enhancing their decision
- Speeding the classification process
- Decreasing human-error occurrence





2) Improved accessibility

3) Cost-Efficiency

4) Early Detection

5) Patients after-care using Medical Chatbot

THANKS!

